Shared-state DSL stress test

Purpose

The current shared-state experiment contains 35 concrete `SharedPrimitive` classes. Many are shallow: they declare a state dictionary, expose one or two write methods, enforce answer constraints, and calculate a view or settlement.

This stress test asks whether those classes can instead be serialized recipes over a small state-machine language. It is intentionally separate from the core EDSL implementation. The executable catalog is `examples/shared_state_dsl_experiment.py`.

First complete result

Every current concrete primitive, plus the proposed `SharedRegister`, has been rewritten as a serializable `Machine` recipe. The prototype validates that:

- every current class has exactly one recipe;
- every effect targets a declared field;
- every input reference is declared by its command;
- the entire 36-recipe catalog serializes to JSON.

The first pass uses nine structural node kinds:

```
type       ref        record
set        set_once   put
append     call       algorithm
```

That result is promising but not sufficient. The recipes also name 16 transition algorithms and 36 pure helper functions. Some of these names merely hide behavior that should be expressed using general collection and arithmetic expressions. Counting only the nine outer node kinds would therefore overstate the simplification.

Proposed minimal kernel

The second-pass target has four parts.

Types

```
Boolean  Number  Text  Choice
Optional  Sequence  Map  Record
```

Types carry serializable constraints such as numerical bounds, allowed choices, record fields, and collection item types.

References

```
constant.<name>
state.<name>
input.<name>
current.<path>
```

`current` paths are checked references to agent, assignment, run, and other execution context. They are data in the serialized recipe, not arbitrary Python attribute access on a worker.

Atomic effects

```
set(field, expression)
set_once(field, expression)
put(map_field, key, expression, once=False)
append(sequence_field, expression)
```

A command applies all its effects atomically against one state version. Counter updates, removals, and nested updates should first be attempted as expressions that produce a new collection value rather than added as new top-level effect types.

Pure expressions

The expression algebra needs literals and records; Boolean and arithmetic operators; conditionals; collection lookup and replacement; and a small set of reducers such as length, sum, mean, median, minimum, maximum, and count-by.

Named payoff functions such as `dictator_payoffs` should not become registered runtime functions. A named recipe should contain the serialized arithmetic formula directly. Otherwise shallow primitive classes have merely become shallow function registrations.

Full catalog rewrite

The table records what each existing class becomes in the first-pass catalog. "Generic" means its state transition can be represented with ordinary typed fields and effects. "Expand" means the first pass used a named helper that the second pass should replace with general expressions. "Built-in" means a substantial, named scientific algorithm may justify a versioned implementation.

Current class	Recipe structure	Second-pass status
`SharedLog`	Sequence plus `append`	Generic

Current class	Recipe structure	Second-pass status
`SharedWorkPool`	Available sequence, claims map, completed map	Expand queue claim
`SharedSignalSchedule`	Signal constants, revealed map, event sequence	Expand indexed reveal
`SharedBinaryMarket`	Quantities, portfolios, trades, settlement	Built-in LMSR calculation
`SharedUltimatumGame`	Four optional fields and two `set_once` commands	Generic
`SharedMoneyRequestGame`	Player-to-request map and payoff formula	Generic
`SharedMatrixGame`	Seat-to-action map and payoff-table lookup	Generic
`SharedRepeatedMatrixGame`	Round-to-action maps and repeated lookup	Expand nested map update
`SharedDictatorGame`	Actor fields, bounded transfer, arithmetic payoff	Generic
`SharedTrustGame`	Two bounded transfers and arithmetic payoff	Generic
`SharedBeautyContest`	Player-to-number map and aggregate formula	Generic
`SharedCommonPoolGame`	Player-to-request map and aggregate formula	Generic
`SharedCentipedeGame`	History sequence, terminal predicate, configured payoff lookup	Expand configured transition table
`SharedMarketEntryGame`	Player-to-Boolean map and congestion formula	Generic
`SharedSealedAuction`	Bid map, hidden view, close-time mechanism	Built-in auction settlement family
`SharedBilateralTrade`	Offer fields, decision field, arithmetic payoff	Generic
`SharedSignalingGame`	Signal fields, decision field, arithmetic payoff	Generic
`SharedNashDemandGame`	Seat-to-demand map and arithmetic payoff	Generic
`SharedVotingGame`	Voter-to-candidate map and count-by view	Generic
`SharedCheapTalkGame`	Message fields, action field, outcome formula	Generic

Current class	Recipe structure	Second-pass status
`SharedPrincipalAgentGame`	Contract fields, effort field, expected-payoff formula	Generic
`SharedCoalitionPool`	Membership maps, capacities, request history	Expand conditional collection update
`SharedBudgetPool`	Remaining amount, funded map, allocation history	Expand bounded atomic arithmetic
`SharedDocument`	Text field and revision sequence	Generic
`SharedCounterMap`	Key-to-count map	Expand collection-wide increment
`SharedMatchPool`	Ranked requests and close-time matches	Built-in matching mechanism family
`SharedDeferredAcceptance`	Ranked requests, capacities, priorities, matches	Built-in deferred acceptance
`SharedDoubleAuction`	Accounts, orders, trades, clearing	Built-in auction clearing family
`SharedResourceBoard`	Assignment maps, capability constraints, attempts	Expand conditional collection update
`SharedAuction`	Bid sequence, maximum, winning index	Generic
`SharedMessageBoard`	Message-record sequence	Generic
`SharedNegotiation`	Turn-record sequence and terminal predicate	Generic
`SharedAgenda`	Proposal sequence, ballot sequence, weighted count-by	Expand generated identifiers
`SharedDelphiPanel`	Response sequence, grouped summaries, convergence predicate	Generic reducers plus formulas
`SharedForecast`	Forecast sequence and weighted mean	Generic reducers plus formulas
proposed `SharedRegister`	Typed map and `put`/`put_once`	Generic foundation

Question composition

Commands declare typed inputs independently of questions:

```
vote = Command(
    inputs={
        "voter": Text(),
        "candidate": Choice(CANDIDATES),
    },
    effects=(
        put(
            state.ballots,
            key=input.voter,
            value=input.candidate,
            once=True,
        ),
    ),
)
```

A recipe factory exposes that command as a method. Binding a question checks its answer contract at construction:

```
survey = Survey([
    candidate,
    election.vote(
        voter=current.agent.name,
        candidate=candidate.answer,
    ),
])
```

The question remains responsible for elicitation and answer validation. The command remains responsible for atomic state transition constraints. Compatible answer shapes can therefore be reused across recipes without making the DSL depend on every concrete EDSL question class.

Named recipes remain useful

Removing concrete primitive classes does not require removing domain names:

```
game = games.ultimatum(stake=100)
market = markets.binary(contract="Event occurs", liquidity=50)
mechanism = matching.deferred_acceptance(capacities, priorities)
```

These factories return ordinary `Machine` recipes. They provide discoverability, domain-specific documentation, and good defaults without creating new execution types.

Boundary for registered algorithms

The DSL should not grow loops, arbitrary callbacks, imports, or unrestricted Python merely to eliminate every built-in. A registered algorithm is justified

when it is:

- scientifically named and independently testable;
- deterministic and atomic;
- expensive or obscure to express with collection reducers;
- explicitly versioned in serialization.

The likely irreducible families after a second pass are:

1. LMSR pricing and settlement;
2. matching mechanisms such as deferred acceptance and serial dictatorship;
3. auction clearing and mechanism-specific settlement.

Centipede transitions, capacity checks, work claiming, counter updates, and payoff formulas should first be expressed with the generic language. If they cannot be represented clearly, that is evidence for one missing general construct—not automatically evidence for another domain class.

Evaluation

The experiment supports the refactor, with a warning. Typed fields, four atomic effects, pure expressions, and a few registered algorithm families appear able to replace most of the 35 concrete classes. The first-pass helper count shows that the expression algebra must be designed and measured carefully. A DSL is smaller only if domain behavior is visible in serialized recipes rather than hidden behind differently named runtime functions.